Predicting Music Listening Behaviour Using Time Series Analysis

A Beginner-Friendly Guide to the Complete Pipeline:
Feature Engineering → Daily Aggregation → SARIMA Forecasting

ML Project Team

March 17, 2026

Abstract

This report explains, step by step, a machine-learning project that analyses real-world music listening data from **Last.fm**. The goal is to understand *when* people listen to music by aggregating raw scrobble logs into daily play-count time series, and then **forecast future daily play counts** using a SARIMA model. The approach is deliberately simple — no embeddings, no SVD, no clustering — just counting plays per day and modelling that count over time. Every concept is introduced from scratch so that a beginner can follow along, reproduce the results, and present them confidently.

Contents

1 Introduction **2**
1.1 What Is This Project About? . . . 2
1.2 Why Does This Matter? . . . 2

2 The Dataset **2**
2.1 Source . . . 2
2.2 Size and Shape . . . 2
2.3 Columns . . . 3
2.4 Per-User Play Counts . . . 3
2.5 A Peek at the Raw Data . . . 3

3 High-Level Pipeline Overview **3**

4 Step 1 — Feature Engineering **4**
4.1 What Is Feature Engineering? . . . 4
4.2 Combining Date and Time . . . 4
4.3 Extracting the Hour . . . 4
4.4 Time Period Bins . . . 4
4.5 Day of the Week and Weekend Flag . . . 5
4.6 Summary of Engineered Features . . . 5

5 Step 2 — Aggregating to a Daily Time Series **5**

5.1 Why Aggregate? . . . 5
5.2 Choosing a User . . . 5
5.3 Code . . . 5
5.4 What Each Line Does . . . 6

6 Step 3 — Stationarity, Decomposition, and ACF/PACF 6
6.1 What Is a Time Series? . . . 6
6.2 Stationarity and the ADF Test . . . 6
6.2.1 What Is Stationarity? . . . 6
6.2.2 Augmented Dickey–Fuller (ADF) Test . . . 6
6.2.3 Differencing . . . 7
6.3 Seasonal Decomposition . . . 7
6.4 ACF and PACF Plots . . . 7
6.4.1 Autocorrelation Function (ACF) . . . 7
6.4.2 Partial Autocorrelation Function (PACF) . . . 7
6.4.3 How to Read ACF/PACF . . . 8
6.5 SARIMA Model . . . 8
6.5.1 What Is ARIMA? . . . 8
6.5.2 Adding Seasonality: SARIMA . . . 8

7 Step 4 — SARIMA Forecasting and Evaluation 8
7.1 Train/Test Split (Temporal) . . . 9
7.2 Evaluation Metric: MAE . . . 9
7.3 Forecasting and Plotting . . . 9

8 Concepts Glossary 9

9 Libraries and Tools Used 10

10 Project File Structure 10

11 Putting It All Together — End-to-End Walkthrough 11

12 Potential Improvements 11

13 Conclusion 12

1 Introduction

1.1 What Is This Project About?

Imagine you have a record of every song a group of people played over the course of a month. This project takes that record and asks two key questions:

1. **When do people listen?** — Do they listen more in the morning or at night? On weekdays or weekends? Are there weekly patterns?
2. **Can we predict future listening?** — Given past daily play counts, can we forecast how many songs a user will play tomorrow or next week?

The approach is intentionally straightforward: aggregate raw listening events into a **daily play-count time series** for a single user, then apply classical time series techniques (stationarity testing, seasonal decomposition, ACF/PACF analysis) to fit a **SARIMA model** that forecasts future activity.

1.2 Why Does This Matter?

Music streaming platforms (Spotify, Apple Music, YouTube Music) use exactly these techniques to:

- Schedule push notifications at the right time of day.
- Predict server load for capacity planning.
- Understand user engagement trends over time.
- Detect changes in listening behaviour (e.g. a user becoming less active).

2 The Dataset

2.1 Source

The data comes from **Last.fm**, a music tracking service that logs every song (called a *scrobble*) a user plays. The dataset is stored in `fm_data.csv`.

2.2 Size and Shape

Property	Value
Total rows (scrobbles)	166,153
Number of users	11
Unique artists	22,823
Unique tracks	67,241
Date range	1 Jan 2021 – 31 Jan 2021 (31 days)

2.3 Columns

Column	Description
`Username`	The Last.fm username (e.g. `Babs_05`).
`Artist`	The artist name.
`Track`	The track (song) name.
`Album`	The album the track belongs to.
`Date`	The calendar date (e.g. `31 Jan 2021`).
`Time`	The clock time (e.g. `23:36`).

2.4 Per-User Play Counts

Not all users are equally active. Here is how the scrobbles are distributed:

Username	Play Count
Babs_05	33,695
franhale	32,712
Knapster01	27,015
eartle	20,966
massdosage	19,015
jonocole	17,230
Orlenay	10,123
isaac	1,780
mremond	1,452
jajo	1,102
lobsterclaw	1,063

2.5 A Peek at the Raw Data

Each row records one listening event:

Username	Artist	Track	Album	Date	Time
Babs_05	Isobel Campbell	The Circus Is...	Ballad of...	31 Jan 2021	23:36
Babs_05	Isobel Campbell	Dusty Wreath	Ballad of...	31 Jan 2021	23:32
Babs_05	The Coral	Sandhills...	Lockdown...	31 Jan 2021	22:56

3 High-Level Pipeline Overview

The entire project flows through five major stages. Each stage feeds into the next:

1. Feature Engineering → **2. Daily Aggregation** → **3. Stationarity Check** → **4. Decomposition + ACF/PACF** → **5. SARIMA Forecasting**

1. **Feature Engineering** — Parse dates/times, create new columns (hour, day of week, time period, weekend flag).
2. **Daily Aggregation** — For a chosen user, count the number of plays per day to produce a daily time series.

3. **Stationarity Check** — Use the Augmented Dickey–Fuller (ADF) test to determine whether the series is stationary; apply differencing if it is not.
4. **Decomposition + ACF/PACF** — Decompose the series into trend, seasonality, and residual; inspect ACF and PACF plots to guide model order selection.
5. **SARIMA Forecasting** — Fit a Seasonal ARIMA model on 80% of the data and forecast the remaining 20%, evaluating with MAE.

4 Step 1 — Feature Engineering

4.1 What Is Feature Engineering?

Raw data rarely comes in a form that machine-learning models can use directly. **Feature engineering** is the process of transforming raw columns into new, more informative columns (called *features*).

4.2 Combining Date and Time

The CSV stores the date and time in separate text columns. We merge them into a single `datetime` object so that Python can do arithmetic on it (e.g. compute differences, extract parts):

```
df['datetime'] = pd.to_datetime(
    df['Date'] + ' ' + df['Time'], dayfirst=True
)
df = df.sort_values('datetime').reset_index(drop=True)
```

`dayfirst=True` tells pandas that the day comes before the month (`31 Jan 2021`, not `Jan 31 2021`).

4.3 Extracting the Hour

From the datetime we pull out the **hour** (0–23):

```
df['hour'] = df['datetime'].dt.hour
```

This single number tells us *when* during the day a song was played.

4.4 Time Period Bins

Hours are fine-grained. For a simpler picture we group them into four **time periods**:

Period	Hours
Morning	05:00 – 11:59
Afternoon	12:00 – 16:59
Evening	17:00 – 20:59
Night	21:00 – 04:59

```python
def time_period(hour):
    if   5 <= hour < 12: return 'Morning'
    elif 12 <= hour < 17: return 'Afternoon'
    elif 17 <= hour < 21: return 'Evening'
    else:                 return 'Night'

df['period'] = df['hour'].apply(time_period)
```

4.5 Day of the Week and Weekend Flag

```python
df['day_of_week'] = df['datetime'].dt.day_name()
df['is_weekend']  = df['day_of_week'].isin(['Saturday', 'Sunday'
    ])
```

Why? Listening habits often differ between weekdays and weekends. Someone might listen to focus music on Monday and party music on Saturday.

4.6 Summary of Engineered Features

Feature	Type	Purpose
`datetime`	Timestamp	Single sortable time column.
`hour`	Integer 0–23	Fine-grained time of day.
`period`	Categorical	Coarse time of day (Morning, etc.).
`day_of_week`	Categorical	Which day (Monday…Sunday).
`is_weekend`	Boolean	Weekday vs. weekend.
`date_only`	Date	Calendar date for daily aggregation.

5 Step 2 — Aggregating to a Daily Time Series

5.1 Why Aggregate?

The raw data contains one row per scrobble (166,153 rows). To perform time series analysis we need a single number per time step. The simplest and most natural choice is to **count the number of plays per day** for one user.

5.2 Choosing a User

We start with the most active user, `Babs_05` (33,695 scrobbles), because a higher play count gives the time series more variation and makes patterns easier to detect.

5.3 Code

```python
user = 'Babs_05'
user_df = df[df['Username'] == user]

# Count plays per day
ts = user_df.groupby(user_df['datetime'].dt.date).size()
```

```
ts.index = pd.to_datetime(ts.index)
ts = ts.asfreq('D').fillna(0)  # fill days with no listens as 0
```

5.4 What Each Line Does

1. **Filter** the dataframe to keep only rows belonging to `Babs_05`.
2. **Group by date** and count the number of rows (scrobbles) per day using `.size()`.
3. **Convert the index** to proper `DatetimeIndex` so that pandas recognises it as a time series.
4. **`asfreq('D')`** ensures every calendar day is present in the index — even days on which the user did not listen. Without this step, missing days would create gaps that confuse time series models.
5. **`fillna(0)`** fills those newly created gaps with zero, meaning "no plays that day".

The result is a `pandas.Series` of length 31 (one value per day in January 2021) with integer play counts.

6 Step 3 — Stationarity, Decomposition, and ACF/-PACF

This step prepares the daily time series for modelling by checking its statistical properties and analysing its structure.

6.1 What Is a Time Series?

A time series is any sequence of data points indexed by time:

$$y_1,\ y_2,\ y_3,\ \ldots,\ y_T$$

In our case y_t = number of songs played on day t. With 31 days of data we have $T = 31$.

6.2 Stationarity and the ADF Test

6.2.1 What Is Stationarity?

A time series is **stationary** if its statistical properties (mean, variance) do not change over time. Most forecasting models require (or work best with) stationary data.

6.2.2 Augmented Dickey–Fuller (ADF) Test

The ADF test is a statistical hypothesis test:

- **Null hypothesis** H_0: The series has a unit root (it is *non-stationary*).
- **Alternative** H_1: The series is stationary.

If the **p-value** < 0.05 we reject H_0 and conclude the series is stationary.

```python
from statsmodels.tsa.stattools import adfuller

result = adfuller(ts)
print(f'p-value: {result[1]:.4f}')
# p < 0.05  ->  stationary (good to go)
# p >= 0.05 ->  difference the series
```

6.2.3 Differencing

If the series is non-stationary we apply **differencing**:

$$y'_t = y_t - y_{t-1}$$

This removes trends. If one round is not enough, we difference again (second-order differencing).

6.3 Seasonal Decomposition

Any time series can be decomposed into three components:

$$y_t = T_t + S_t + R_t$$

- T_t — **Trend**: the long-term direction (increasing, decreasing, or flat).
- S_t — **Seasonality**: repeating patterns at a fixed period (e.g. weekly).
- R_t — **Residual**: the leftover noise.

```python
from statsmodels.tsa.seasonal import seasonal_decompose

result = seasonal_decompose(ts, model='additive', period=7)
result.plot()
```

We choose `period=7` because listening patterns are expected to repeat weekly (e.g. less listening on workdays, more on weekends).

6.4 ACF and PACF Plots

6.4.1 Autocorrelation Function (ACF)

The ACF measures the correlation between y_t and y_{t-k} for various lags k. It answers: "How much does today's value depend on past values?"

6.4.2 Partial Autocorrelation Function (PACF)

The PACF measures the correlation between y_t and y_{t-k} *after removing the effects of all intermediate lags*. It helps isolate the *direct* influence of lag k.

```python
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

plot_acf(ts, lags=30)
plot_pacf(ts, lags=30)
```

6.4.3 How to Read ACF/PACF

Pattern	ACF	PACF
AR(p) process	Tails off gradually	Cuts off after lag p
MA(q) process	Cuts off after lag q	Tails off gradually
ARMA(p, q)	Tails off	Tails off

These patterns guide the choice of model orders (see SARIMA below).

6.5 SARIMA Model

6.5.1 What Is ARIMA?

ARIMA stands for **A**uto**R**egressive **I**ntegrated **M**oving **A**verage. It has three parts controlled by the order (p, d, q):

- p (**AR** — Auto-Regressive): Number of past values used as predictors.

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \epsilon_t$$

- d (**I** — Integrated): Number of times the series is differenced to make it stationary.
- q (**MA** — Moving Average): Number of past forecast errors used as predictors.

$$y_t = c + \epsilon_t + \theta_1 \epsilon_{t-1} + \cdots + \theta_q \epsilon_{t-q}$$

6.5.2 Adding Seasonality: SARIMA

SARIMA extends ARIMA with a seasonal component. Its full specification is:

$$\text{SARIMA}(p, d, q) \times (P, D, Q, s)$$

where (P, D, Q) are the seasonal AR, differencing, and MA orders, and s is the seasonal period (7 for weekly).

```python
from statsmodels.tsa.statespace.sarimax import SARIMAX

model = SARIMAX(ts, order=(1,0,1),
                seasonal_order=(1,0,1,7))
result = model.fit()
print(result.summary())
```

In our case:

- $(p, d, q) = (1, 0, 1)$: one AR lag, no differencing, one MA lag.
- $(P, D, Q, s) = (1, 0, 1, 7)$: one seasonal AR lag, no seasonal differencing, one seasonal MA lag, weekly seasonality.

7 Step 4 — SARIMA Forecasting and Evaluation

This section covers the core modelling step: fitting a SARIMA model on a training portion of the daily time series and evaluating its forecast on a held-out test set.

7.1 Train/Test Split (Temporal)

For time series data we **never shuffle**. We split chronologically:

```
split = int(len(ts) * 0.8)
train, test = ts[:split], ts[split:]
```

The first 80% of days are used for training; the last 20% for testing. This prevents **data leakage** — using future data to predict the past.

7.2 Evaluation Metric: MAE

Mean Absolute Error measures average prediction error in the same units as the data (plays per day):

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

```
from sklearn.metrics import mean_absolute_error
print(f'MAE: {mean_absolute_error(y_test, y_pred):.2f}')
```

A lower MAE means better predictions.

7.3 Forecasting and Plotting

The final step is to generate predictions on the test set and visualise them:

```
model = SARIMAX(train, order=(1,0,1),
                seasonal_order=(1,0,1,7)).fit()
forecast = model.forecast(steps=len(test))

pd.DataFrame({'actual': test, 'forecast': forecast}).plot()
```

This produces a plot with:

- The **training data** (historical context).
- The **actual** test values (ground truth).
- The **forecast** line (model predictions).

8 Concepts Glossary

For quick reference, here is every key concept used in this project:

Concept	One-Line Explanation
Feature Engineering	Creating new informative columns from raw data.
Time Series	A sequence of data points indexed in time order.
Stationarity	Statistical properties (mean, variance) constant over time.
ADF Test	Statistical test for stationarity (Augmented Dickey–Fuller).
Differencing	Subtracting consecutive values to remove trends: $y'_t = y_t - y_{t-1}$.
Seasonal Decomposition	Splitting a series into trend, seasonality, and residual.
ACF	Autocorrelation function — correlation of a series with its own lagged values.
PACF	Partial autocorrelation — direct correlation at lag k, removing intermediate effects.
AR (Auto-Regressive)	Using past values of the series as predictors.
MA (Moving Average)	Using past forecast errors as predictors.
ARIMA	Model combining AR, differencing, and MA components.
SARIMA	ARIMA extended with seasonal AR, differencing, and MA terms.
MAE	Mean Absolute Error — average prediction error magnitude.
Train/Test Split	Dividing data chronologically to evaluate model performance.
Data Leakage	Accidentally using future information during training.
`asfreq('D')`	Ensures every calendar day appears in the index (fills gaps).
`fillna(0)`	Replaces missing values with zero (no plays that day).

9 Libraries and Tools Used

Library	Version	Role in This Project
pandas	3.0.1	Data loading, manipulation, grouping, time series indexing (`asfreq`, `fillna`).
NumPy	2.4.2	Numerical operations on arrays.
scikit-learn	1.8.0	`mean_absolute_error` for evaluation.
statsmodels	—	ADF test, seasonal decomposition, ACF/-PACF plotting, SARIMAX model.
matplotlib	3.10.8	All plots and charts.

10 Project File Structure

File / Folder	Description
`fm_data.csv`	Raw Last.fm scrobble data (166,153 rows).
`info.txt`	Step-by-step plan for the time series analysis pipeline.
`requirements.txt`	Python dependencies with pinned versions.
`project_report.tex`	This report (LaTeX source).
`time_feature_engineering.ipynb`	Notebook for feature engineering exploration.
`test.ipynb`	Notebook: time series analysis and SARIMA forecasting.
`test2.ipynb`	Notebook: additional forecasting experiments.

11 Putting It All Together — End-to-End Walkthrough

Step 1. **Load the CSV** and parse `Date + Time` into a single `datetime` column.

Step 2. **Engineer features**: extract hour, time period, day of week, and weekend flag for exploratory analysis.

Step 3. **Pick a user** (e.g. `Babs_05`) and **count plays per day** to create a daily time series.

Step 4. **Fill gaps**: use `asfreq('D').fillna(0)` so every calendar day has a value.

Step 5. **Check stationarity** with the ADF test; difference the series if the p-value ≥ 0.05.

Step 6. **Decompose** into trend, seasonality (period = 7), and residual.

Step 7. **Inspect ACF/PACF** plots to determine appropriate SARIMA orders (p, d, q) and (P, D, Q, s).

Step 8. **Split chronologically**: first 80% for training, last 20% for testing.

Step 9. **Fit a SARIMA model** on the training set with `SARIMAX(train, order=(1,0,1), seasonal_order=(1,0,1,7))`.

Step 10. **Forecast** the test period and compute MAE.

Step 11. **Plot actual vs. forecast** to visually assess model quality.

12 Potential Improvements

- **More data**: One month (31 days) is short. With several months the model could learn longer-term seasonal and monthly patterns more reliably.
- **Auto-ARIMA**: Use `pmdarima.auto_arima` to automatically search for the best $(p, d, q)(P, D, Q, s)$ orders instead of manually reading ACF/PACF plots.
- **Prophet / Neural models**: Facebook Prophet or LSTM networks can capture complex non-linear patterns.
- **Multi-user modelling**: Instead of one user at a time, model all users jointly or compare SARIMA fits across users.

- **Exogenous variables**: Feed day-of-week, time-period proportions, or external data (e.g. weather, holidays) as regressors into SARIMAX.
- **Cross-validation**: Use rolling-window or expanding-window cross-validation instead of a single 80/20 split for more robust evaluation.

13 Conclusion

This project demonstrates a complete, beginner-friendly time series analysis pipeline on real music listening data:

1. We **engineered features** from raw timestamps to understand listening patterns by hour, time period, and day of week.
2. We **aggregated** raw scrobble events into a **daily play-count time series** for a single user.
3. We checked **stationarity** using the ADF test and **decomposed** the series into trend, seasonality, and residual components.
4. We used **ACF and PACF** plots to guide the selection of model orders.
5. We fit a **SARIMA model** on the training data and **forecast** future daily play counts, evaluating accuracy with MAE.

The pipeline is deliberately simple — no embeddings, no dimensionality reduction, no clustering — just counting plays and modelling that count over time. This makes every step easy to understand, reproduce, and extend.